Raman Kapoor

PROJECT PROPOSAL · LFX MENTORSHIP 2026 TERM 2

unchangedraman@gmail.com
github.com/unchangedraman
Gwalior, India · IST (UTC+5:30)

To: Charalampos Mainas (@cmainas) & Anastassios Nanos (@ananos) — Nubificus / urunc 14 May 2026

Project: CNCF · urunc · Improve lifecycle management of sandbox monitors **Upstream:** urunc-dev/urunc#112

1. About me

I am Raman Kapoor — a B.Tech (Information Technology) graduate from ABV-IIITM Gwalior (class of 2026), currently working at Juspay, India's largest payments orchestration platform, on low-level Linux infrastructure. My most recent landed piece there was leading the migration of our block-storage tier from CEPH to Lightbits NVMe/TCP — same workload at 48K TPS versus 37K TPS on roughly a tenth of the hardware (four nodes instead of forty), with active-active Barbican HA on a shared backend. Before that I shipped a thread-local and server-local cache layer in Rust on our hot APIs that cut cross-node calls by ~4% at peak, and a Go-based Shamir-secrets distribution broker on top of OpenBao that removed our reliance on AWS KMS for key delivery. Right now I am on the bare-metal datacenter side of the stack — writing the secure-erase command for our internal Rust CLI that drives MAAS-managed hosts through their lifecycle.

My interest in monitors and hypervisors started earlier than the day job, with a 32-bit i386 kernel I wrote from scratch in C++ during undergrad — bootstrap, protected mode, PIC and ISR wiring, the GDT. That project is what made me want to work on the userspace side of the same problem; urunc and this LFX project are exactly that.

2. What I have been working on

2.1 Open-source contributions to Namma Yatri (Haskell, production)

The work I am proudest of, and the closest match to what this LFX project will involve in terms of *shipping upstream code in a large unfamiliar codebase with maintainer review*, is my contribution to Namma Yatri — the open-source ride-hailing platform that runs in production across India. Between November 2025 and January 2026 I shipped three merged PRs into the org:

- `feat` nammayatri/nammayatri#13442 — Special-zone driver-payout scheduling system. A distributed, event-driven scheduler serving roughly 10,000 rides per day; cut driver payout latency from weekly cycles down to T+2 hours under production traffic.
- `feat` nammayatri/nammayatri#13452 — Exposed the resulting driver-payout state to the driver-app frontend through the API layer.
- `feat` nammayatri/shared-kernel#1108 — PaytmEDC payment-flow integration in the shared kernel — config management, checksum generation, API interaction. Went through several review iterations before merge.

The technically interesting one is #13442. Driver payouts for special-zone rides (airports, stations) are computed per-ride but were previously paid out only on a weekly batch cycle — a long latency for drivers, and operationally fragile when the batch failed. The new system schedules per-ride payouts at

T+2 hours after ride completion, which means roughly 10,000 scheduled jobs in flight at any given moment under load. The constraints made the design non-trivial:

- **Exactly-once execution.** A payout that fires twice is real money lost; a payout that never fires is a real driver not paid. The scheduler had to be exactly-once under broker restart, app restart, and partial-failure scenarios.
- **No latency on the ride-booking hot path.** Scheduling a payout is a side-effect of completing a ride; the ride-completion API could not block on scheduling state.
- **Idempotent state transitions.** A retry of the payout-fire call had to be safe — same input must yield the same end state, not double-debit.

The shape I landed on is a Redis-backed delayed-job scheduler keyed on ride-id, with per-ride distributed locks taken at fire time and an idempotency token persisted before the actual payout call. Scheduling is O(1) on the hot path (a single Redis sorted-set insert with the fire-time as score), and the worker pool polls the sorted set to find due jobs. The per-ride lock ensures only one worker fires a given payout even if two pick it up simultaneously, and the idempotency token means a retry after a partial failure is safe. Working on a Haskell codebase with no prior production Haskell experience, designing the failure model with the team, and getting to merged three times in a row, is the closest thing I have done to what this LFX project will involve — and it is the muscle I am leaning on for that reason.

2.2 Juspay — production infrastructure work

Beyond the storage migration touched on in §1, the work at Juspay most relevant to this LFX project lives in two places: the identity-and-secrets layer running on our OpenStack tenancy, and the bare-metal datacenter-lifecycle CLI I am currently building.

2.2.1 Identity and secrets on OpenStack + OpenBao

We replaced a heavy AWS KMS dependency with an in-house identity-and-secrets layer running on OpenStack VMs and OpenBao. A few design choices shaped most of what I built:

- **Identity derived from the network, not from a long-lived token.** A VM authenticating to the broker presents an IMDS-attested claim — its identity *is* the network position the orchestrator placed it at, signed by the platform. There is no static credential on disk an attacker can steal. The broker validates the attestation and issues a short-lived RS256-signed token scoped to what that VM is allowed to read.
- **Shamir secret-sharing for the root key.** No single broker node holds the full root; it is split into *m*-of-*n* shards distributed across the cluster, and reassembled in memory only when needed. A single compromised node yields no useful secret material.
- **Memory-only keys, never on disk.** Keys live in mlock'd pages; the broker process refuses to start if the OS supports core dumps unsanitized.
- **HA via Raft.** Brokers run a Raft cluster across availability zones, so the identity service tolerates $N-1$ broker failures without an outage — the same posture etcd or CockroachDB take, applied to secrets.
- **VM-side SDK in Python (`osvault`)** for the bootstrap flow — handles attestation, token refresh, leasing, and audit. The broker itself is Go.

The reason I am calling it out for this proposal: it is the closest thing I have at Juspay to what urunc's monitor-control plane needs to be — an API-driven lifecycle service that must be correct under partial failures, talks over Unix-domain or HTTP sockets, and cannot lose state across restarts. The error-domain modelling work I did there (token-not-yet-issued vs token-expired vs broker-quorum-lost vs network-partitioned) is the same shape I expect to do around socket lifecycle for QMP and the Firecracker API.

2.2.2 Bare-metal datacenter lifecycle — `namma-cli` and the `wipe-disk` work in flight

My current piece of work at Juspay is on `namma-cli` — a Rust CLI that drives the lifecycle of bare-metal hosts in our datacenter, sitting on top of MAAS for hardware control. Hosts are described declaratively in site specs; `namma-cli` reconciles spec → actual state. The architectural shape is similar to Cluster API, scoped to bare metal rather than VMs.

I am working on the `wipe-disk` command — secure-erase of all drives on a host before reprovisioning, using NVMe Sanitize where the controller advertises it and falling back to `hdparm --security-erase`. The CLI orchestrates: power-on via IPMI, MAAS commission into an ephemeral OS, push the wipe script over SSH, run it, wait for `poweroff -f`, MAAS releases the machine. End-to-end across a real bare-metal pipeline.

The piece of that work most worth surfacing here is an engineering-discipline find. The original `Maas::wipe_disk` code matched the SSH-exec result with a broad `Err(_)` arm and treated *any* SSH error as the legitimate "host poweroff disconnect," then returned `Ok(())`. In practice that silently masks auth failures, connection-refused, network-unreachable, and command-errors before the wipe script ever ran — every one of them returning `Ok` from a destructive operation that had not actually executed. For `wipe-disk` specifically, that is the worst possible failure mode: silent false-success on the operation that nukes customer data. I caught this on a local end-to-end test, narrowed it to the match arm, and proposed a discrimination heuristic — if the remote command produced no stdout before the disconnect, treat it as an error rather than as expected poweroff. This is the same kind of error-domain discipline the urunc monitor-control plane will need around socket open/close timing, monitor-process death, and the difference between "QMP socket closed because we asked for `quit`" and "QMP socket closed because the monitor died."

2.2.3 Hot-path caching in Rust

Already touched on in §1. The short version: a thread-local plus server-local cache layer in front of hot read APIs cut cross-node calls by roughly 4% at peak with strict cache-safety boundaries — only immutable, request-scoped data, with a static analyzer enforcing the boundary at compile time so the boundary cannot quietly erode.

2.3 Independent project: PostgreSQL on Kubernetes / OpenStack

A 5-layer nested virtualization stack on a single i9-14900 box: bare-metal Linux at the bottom, KVM running an OpenStack tenancy on top, Kubernetes running inside that, CloudNativePG inside Kubernetes, Postgres inside CNPG. The point of stacking it this deep was to validate the same shape Juspay's production stack runs at, on hardware I could actually instrument. The work was equal parts performance tuning, lifecycle automation, and identity plumbing.

Performance tuning. The inner-VM Postgres saw write-TPS climb roughly 50% after two pieces of work — NUMA-aware CPU pinning of the Postgres workers to the same socket as the L1 hypervisor exposing the disks, and 16 GB of hugepages dedicated to the inner VM. The latter mattered specifically because Postgres' buffer pool was thrashing the TLB at default page sizes; 2 MB pages cut TLB misses enough that the syscall cost on every page fault went away. Tuning at a layer of virtualization you do not control is mostly about *measuring* what is actually happening, which is the same skill set #389 needed.

Lifecycle automation. Cluster API for OpenStack (CAPO) provisions VMs declaratively; CloudNativePG is the Kubernetes operator that runs the Postgres cluster. End-to-end: a single `kubectl apply` brings up a HA Postgres cluster (one primary + N replicas, synchronous WAL streaming) in ~12 minutes from cold. Failover is automatic — CNPG runs a leader-election loop, demotes the failing primary, promotes a healthy replica, and rewires the service endpoint. Tested under primary-VM kill and primary-VM network partition; recovery to write-ready in roughly 7 seconds with RPO=0 (no acknowledged write lost, because the synchronous stream guarantees the standby has it before the primary acks the client).

Identity plumbing. A Go-based OpenBao broker plus the `osvault` Python SDK on the VM side delivers secrets at bootstrap — the VM proves its OpenStack-assigned identity via IMDS, the broker validates it, the SDK fetches the secrets it is allowed to. Same shape as the identity work I described in §2.2.1, just applied to a smaller cluster I could rebuild from scratch in an afternoon.

Why this is in the proposal: it is not directly urunc, but it is the same shape of problem — API-driven lifecycle of stateful infrastructure on top of VMM-based isolation, where the layers above the VMM care a lot about what state the VMM is in and need a reliable channel to ask. That is exactly what the urunc monitor-lifecycle work is about.

2.4 BREAK OS — a 32-bit i386 kernel from scratch

A small monolithic kernel I wrote in C++ during undergrad — bootstrap from the boot sector, protected-mode transition, 64 KB GDT for memory segmentation and privilege-level isolation, PIC initialization, the ISR / IRQ table wiring, and a tight interrupt path that lands at roughly 0.1 ms response latency. Including it here for two specific reasons.

First: the LFX project is essentially the userspace side of the same world. Everything urunc does is on top of a process that talks to a hypervisor that runs a guest that, on x86_64, ultimately uses the same 16550 UART at I/O port `0x3F8` that this kernel does. When I read in §2.5 about "per-byte PIO traps at port `0x3F8`," I am reading about the same hardware register I programmed in BREAK OS — which makes the urunc debugging feel less like archaeology and more like familiar territory.

Second: writing a kernel from scratch is the experience that taught me how to operate at the layer where this LFX project lives. There are no helpful libraries, no managed runtime, no friendly error messages — there is what the hardware does, what the manual says, and what your code makes happen between those two. The same posture is what tracing PIO exits in #389 needed, and it is what the QMP socket-lifecycle work will need.

2.5 urunc so far — what I have learned through issue #389

A few weeks ago I picked up urunc-dev/urunc#389 — high host CPU caused by console output in running containers. A prior contributor had attributed the regression to `bufio.Scanner` in `cmd/urunc/log_forward.go`. My own profiling showed that path is not hot during guest execution: that scanner only handles the parent-child JSON log pipe during `urunc create`; once `syscall.Exec()` runs, the VMM manages stdio directly. The real cost lives one layer deeper.

I wrote a small KVM-exit and throughput sampler — `bench-console.sh`, which ananos has since indicated they would like to fold into urunc's CI as a regression check — and traced the regression to per-byte PIO traps on the 16550 UART at port `0x3F8`. The `-serial stdio` baseline costs roughly 1.16 M PIO exits per 5-second window, with QEMU's `address_space_write` dominating the host flame graph. A simple `virtio-console` swap drops PIO exits to ~0 and lifts throughput from 2,016 to 73,378 lines/sec (about 36×), closing the runc-vs-urunc gap on this workload from roughly 407× down to ~11×.

The thread has since covered: how to differentiate `virtio-console` from `serial` without breaking Linux kernels built without `CONFIG_VIRTIO_CONSOLE` (my first proposal, a capability-intersection design borrowed from urunc's rootfs path, was constructively rejected by @cmainas for exactly that reason); the upcoming pivot from the `com.urunc.unikernel.cmdline` annotation to a `kernelArgs`-style annotation that lets users specify boot parameters at build time; and an invitation from @ananos to draft a blog post on the investigation for the Nubificus / CNCF blog.

Through that work I have gained working familiarity with:

- urunc's monitor invocation path — `pkg/unikontainers/hypervisors/qemu.go`, `BuildExecCmd`, and the `syscall.Exec`-style handoff in `unikontainers.go`.
- The distinction between urunc's parent-child JSON log pipe (consumed by `bufio.Scanner` before `syscall.Exec`) and the VMM-managed stdio path that takes over after.
- The QEMU + KVM monitor surface — what costs PIO exits at the 16550 UART, how virtio devices change the picture, where the cost actually lives.
- urunc's Linux unikernel path — `pkg/unikontainers/unikernels/linux.go`, the boot-params construction, the urunit handshake.
- The `VMM` interface in `pkg/unikontainers/types/types.go` and how each monitor (QEMU, Firecracker, HVT) implements it today.
- An empirical debugging posture for urunc — `/sys/kernel/debug/kvm/io_exits`, `perf` flame graphs, and a small reproducer image, rather than guessing.

3. The project — monitor lifecycle (urunc#112)

3.1 The problem

urunc today shells out to QEMU or Firecracker with a long `-x ... -y ...` command string. That is enough to *start* a sandbox, but everything after that — checking what the guest is doing, hotplugging a device, asking the monitor to shut down gracefully, attaching a console after the fact — is either impossible or has to be approximated by signalling the host process and hoping for the best. Both QEMU and Firecracker already expose a much better interface for this: QEMU has the QEMU Machine Protocol (QMP, line-delimited JSON over a Unix socket), and Firecracker exposes a REST-style API

socket. Issue #112 is exactly this gap — urunc has CLI for spawn but no socket-based control plane for the lifecycle that follows.

The cost of not having that surface shows up in real places. Graceful pause/resume is not really possible. Hotplug is not really possible. The urunc-side state machine has to infer monitor state from process exit codes instead of asking the monitor directly. And each new monitor we add (cloud-hypervisor, Dragonball, etc.) is a fresh integration with no shared abstraction.

3.2 My approach

I see this project as two pieces that have to land together. The first is a **monitor-agnostic control interface inside urunc** — an internal Go interface that captures the operations urunc actually needs (probe, query state, send shutdown, hotplug net/block, attach console). The existing `VMM` interface is the natural home; today it carries `BuildExecCmd`, `Stop`, `Path`, `UsesKVM`, `SupportsSharedfs`, `Ok`, and `PreExec` for HVT seccomp. The new surface extends it without breaking those. The second piece is the **per-monitor implementations** that translate the interface into QMP for QEMU and into the Firecracker API for Firecracker, plus a small fallback for monitors that do not expose a socket (so the existing CLI-only flow keeps working for HVT and similar).

For QEMU, I would generate `-qmp unix:<sock>,server,nowait` as part of the existing `BuildExecCmd` flow. After spawn, the sequence is: read the greeting, send `qmp_capabilities`, then any of the operational commands the interface exposes. The QMP commands I would implement in order of project priority are `query-status` (probe + state machine), `quit` (graceful shutdown, replaces `killProcess`), `stop` / `cont` (pause/resume), `device_add` / `device_del` (hotplug for virtio-net and virtio-blk first), `chardev-add` / `chardev-remove` (console attach/detach), and `query-vcpus` / `query-block` for introspection.

For Firecracker, the existing `firecracker-go-sdk` already wraps the API socket; the work is mostly wiring urunc's interface methods to `PauseVM` / `ResumeVM` / `PatchDrive` / `PatchMMDS` / `GetMachineConfig`, and deciding which subset we actually expose at the urunc layer. Firecracker's API surface is smaller than QEMU's, so the interface needs capability flags — the same shape urunc already uses for `SupportsSharedfs`.

3.3 Decision points the design doc will resolve

I have a working position on each of these, but I expect the design doc and your review to settle them — these are not unilateral decisions and I want to be explicit that I know that. On a related note: I have already been through one design-discussion round with @cmainas on this codebase. My first proposal for the virtio-vs-serial console choice (capability-intersection, borrowed from urunc's rootfs path) was constructively rejected because Linux kernels can be built without `CONFIG_VIRTIO_CONSOLE`. The feedback shape from that round is what I expect from this project's design-doc review, and I have re-set my priors accordingly.

> **Decision 1 · QMP wire types — hand-roll vs adopt a library**
> Either declare our own Go structs for the small subset of QMP messages we use, or pull in `github.com/digitalocean/go-qemu` (or similar). Default position: hand-roll, because the subset is small and the third-party libraries pull in a lot of code we will not use. Open to evidence otherwise.

> **Decision 2 · Extend `VMM` vs introduce a sibling `MonitorControl` interface**
> Either grow the existing `VMM` interface with the new methods, or keep `VMM` as-is (spawn/exec concerns) and add a parallel `MonitorControl` interface for post-spawn operations. Default position: sibling interface, because spawn and runtime-control have different lifetimes (the runtime interface only becomes valid after the socket handshake succeeds).

> **Decision 3 · Socket location and cleanup ownership**
> Per-container subdirectory under urunc's state dir, removed on container delete. Default position: yes, but who owns the cleanup if urunc crashes between socket open and use? The fallback recovery semantics need a written policy.

> **Decision 4 · Opt-in vs default-on**
> Should the new control path be default-on for supported monitors, or opt-in via an OCI annotation initially while we burn it in? Default position: opt-in for the first cut so we do not break anyone's existing flow, default-on once the integration tests are green.

3.4 What I will explicitly not do

- I will not rip out the existing `BuildExecCmd` / `syscall.Exec` shape. The new socket-based control is layered on top, not a replacement.
- I will not also rewrite the seccomp handling (`PreExec` for HVT) or the monitor-rootfs work in the same project. Those are adjacent but big enough to be their own conversations.
- I will not add a new monitor (cloud-hypervisor, Dragonball, etc.) inside the LFX scope. The interface should make adding one later easier — that is a different person's first PR.
- I will not introduce a new dependency for QMP unless the design doc justifies it.

3.5 What I will prove out before merging

- The QMP-controlled QEMU path can spawn, query state, gracefully shut down, and hotplug a virtio-net device end-to-end, with the existing nginx-on-unikraft integration tests still green.
- The Firecracker path can do the same subset.
- A simple benchmark — same workload as my #389 work, but measuring monitor-control overhead (round-trip count, socket I/O cost). The design doc gets a real number, not just "it works."
- The fallback path (CLI-only, no socket) still works for every monitor that uses it today.

3.6 Stretch goals — if the core scope lands ahead of schedule

The core deliverable (design doc + QEMU/Firecracker implementation + tests) is what I am committing to. If we land it ahead of the 12-week window, here is what I would propose as the next moves, in priority order — all of them are explicitly out-of-scope unless time allows and the mentors approve:

- **Live console attach over QMP `chardev-add`.** A `urunc attach <container>` command that opens an interactive console on a running sandbox via the new control plane. Directly closes the gap that issue #389 exposed about the cost of always-on stdio.

- **QMP event subscription.** Have urunc *subscribe* to QMP events (`SHUTDOWN`, `RESET`, `STOP`, `BLOCK_IO_ERROR`) instead of polling, and surface them into urunc's container state machine. Cleaner state tracking, less polling overhead.
- **Cloud-hypervisor integration as a third backend** — only as proof that the `MonitorControl` interface generalizes. Not the deliverable, but a useful sanity check on the abstraction.
- **Wire the `bench-console.sh` harness from #389 into CI** with two-metric assertions (lines/sec floor + PIO-exits ceiling), so regressions like the one #389 chased are caught automatically.

4. Plan — 12 weeks (8 Jun – 31 Aug 2026)

I plan to break the implementation phases into multiple small PRs rather than one large one, so review can happen in parallel with development. Weekly check-ins with the mentors plus an open draft PR for the design doc as soon as it is started.

WK	PHASE	CONCRETE DELIVERABLE
1–2	Onboarding + design doc, draft 1	Re-read `pkg/unikontainers/hypervisors/`. Author a "current state of monitor lifecycle in urunc" gist for mentors to correct. Open the draft design doc PR proposing the `MonitorControl` interface, QMP + Firecracker mapping, socket lifecycle, and resolving decisions 1–4.
3	Design doc revisions, sign-off	Address review comments. Tag the doc as ready once mentors sign off. The interface signature and dependency decision are settled before any production code lands.
4–5	QEMU QMP — MVP then full lifecycle	Wire `-qmp` into QEMU `BuildExecCmd`. Implement capability handshake, `query-status`, `quit` first (small PR). Follow with `cont` / `stop`, `device_add` / `device_del` for virtio-net and virtio-blk, error handling for unsupported commands. Unit tests for the wire codec.
6–7	Firecracker API + state-machine integration	Implement the same `MonitorControl` surface against `firecracker-go-sdk` — pause/resume, drive patch, network patch where applicable. Plumb the control path through `unikontainers.go` so urunc's state machine uses it. Confirm the CLI-only fallback still works.
8–9	Tests, CI, hotplug demo	Integration tests in the existing layout. Extend `bench-console.sh` to time QMP round-trip cost. CI workflow update. End-to-end demo: start a unikraft-nginx sandbox, hotplug a second virtio-net device, verify connectivity, document the recipe.
10	Bug-fix + user-facing docs	Review-comment cleanup across all open PRs. User docs: how to talk to a urunc sandbox's QMP socket, how to opt into the new control path, what guarantees the interface gives.
11–12	Buffer + final clean-up + blog post	Reserved for slippage. If none, take a follow-up issue from the mentors' backlog. Mentee blog post for the CNCF / Nubificus blog covering the architecture and the numbers.

4.1 Risk register and mitigations

Twelve weeks is short. I am being explicit about what could slip and how I would handle each:

> **Risk · Design doc takes longer than two weeks to land.**
> Mitigation: I begin the design doc immediately after onboarding and keep it as an open draft PR from week 1, so feedback can land asynchronously. If week 3 still has open structural questions, I deliberately pick the simpler of the two options to unblock the QEMU MVP and revisit the harder one mid-implementation, rather than blocking on consensus.

> **Risk · QMP socket lifecycle edge cases (monitor dies between open and use, partial writes, double-close) are noisier than the design doc accounts for.**
> Mitigation: I plan a small fault-injection harness in week 4 that exercises monitor death at each step of the handshake; surprises surface there rather than in the integration tests.

> **Risk · Firecracker SDK changes shape mid-project.**
> Mitigation: pin the SDK version in `go.mod` at the start of week 6, watch the SDK release notes weekly, and only upgrade inside the LFX window if a fix we need lands upstream.

> **Risk · The existing integration test suite is slower to run than expected and adding QMP tests blows up CI time.**
> Mitigation: I budget a half-week (part of week 8) to profile and trim the test suite; precedent for that kind of work in §2.5 — I already use `perf` and KVM debugfs to find what is actually slow.

5. Pre-application work — in flight before May 19

Between now and the application deadline I am landing the following in the proposal repo (github.com/unchangedraman/proposal) so you can see the work, not just read the plan:

- A small standalone Go program that opens a Unix socket to a `qemu-system-x86_64` instance, sends `qmp_capabilities`, then `query-status` and `quit`. ~100 lines, intentionally throwaway — just to prove the wire shape and surface the rough edges.
- A second, equally small Go program against the Firecracker API socket using `firecracker-go-sdk`, doing `GetMachineConfig` and `PauseVM` / `ResumeVM` on a stopped microVM.
- A one-page sketch (`docs/design-sketch.md`) of what the `MonitorControl` interface signature would look like and where it slots into `pkg/unikontainers/hypervisors/types.go`. Not a finished design — a starting point for the actual design doc in weeks 1–2.
- A short note on the `bench-console.sh` harness extension I would propose for CI — two-metric assertions (lines-per-sec floor and PIO-exits-per-window ceiling), reading from `/sys/kernel/debug/kvm/` exactly as the original script does, so the regression that #389 chased is caught automatically going forward.

None of this is part of the LFX commitment; it is what I am doing this week regardless, because the only way to know whether the design is right is to write a little of it. The proposal repository (github.com/unchangedraman/proposal) is where these artifacts will land as they are written, alongside this PDF.